

ACTIVITY 1 – ARRAYS

A. Student Marks Management

Objective

Practice array creation, input, traversal, and basic computations.

Problem Statement

Write a program that stores the marks of **N students** in an array.

The program should:

1. Read N.
2. Store the marks in an array.
3. Display all marks.
4. Find and display:
 - a. Highest mark
 - b. Lowest mark
 - c. Average mark

Sample Input

78 92 65 81 89

Sample Output

Marks:

78 92 65 81 89

Highest = 92

Lowest = 65

Average = 81

Code:

```
#include <stdio.h>
int main() {
    int n,a[10],i;
    int high=0,low=0,sum=0;
    float avg;
    printf("Enter How many Students: ");
    scanf("%d",&n);
    printf("Enter Marks\t:");
    for(i=0; i<n; i++)
        scanf("%d",&a[i]);
    printf("Marks are: ");
    for(i=0; i<n; i++)
        printf("%d\t", a[i]);
    high = a[0];
    low = a[0];
    for(i=0; i<n; i++) {
        if(a[i]>high)
            high = a[i];
        if(a[i]<low)
            low = a[i];
        sum += a[i];
    }
    avg = (float)sum/n;
    printf("\nHighest = %d", high);
    printf("\nLowest = %d", low);
    printf("\nAverage = %.1f", avg);
    return 0;
}
```

Output:

```
Output
Enter How many Students: 3
Enter Marks :55 73 69
Marks are: 55 73 69
Highest = 73
Lowest = 55
Average = 65.7
```

B. Insert an Element into an Array

Objective

Learn how insertion works by shifting elements.

Problem Statement

Given an array of integers, insert a new element at a specified position.

Assume the array has enough extra space to accommodate the new element.

Input Format

- Size of array N
- N elements
- Position (0-based index)
- Element to insert

Sample Input

```
5
10 20 30 40 50
2
25
```

Sample Output

```
Array after insertion:
10 20 25 30 40 50
```

Constraints

- Position must be between 0 and N.
- If invalid, print

Invalid Position

Code:

```
#include <stdio.h>
int main() {
    int n,a[10],i,pos,ele;
    printf("Enter Size of the Array: ");
    scanf("%d",&n);
    printf("Enter Array Elements:");
    for(i=0; i<n; i++)
        scanf("%d",&a[i]);
    printf("Enter the position at which element is inserted: ");
    scanf("%d",&pos);
    printf("Enter the Element to be inserted: ");
    scanf("%d",&ele);
    n++;
    for(i=n; i>pos; i--)
        a[i-1] = a[i-2];
    a[pos-1] = ele;
    printf("After insertion Array Elements are: ");
    for(i=0; i<n; i++)
        printf("%d\t",a[i]);
    return 0;
}
```

Output:

Output

```
Enter Size of the Array: 5
Enter Array Elements:10 20 30 40 50
Enter the position at which element is inserted: 3
Enter the Element to be inserted: 33
After insertion Array Elements are: 10 20 33 30 40 50
```

C. Delete an Element from an Array

Objective

Understand deletion by shifting elements to the left.

Problem Statement

Given an array and a value to delete, remove the **first occurrence** of the value.

If the value is not present, print an appropriate message.

Sample Input

```
6
15 20 35 20 40 50
20
```

Sample Output

```
Array after deletion:
15 35 20 40 50
```

Sample Input 2

```
5
5 8 9 1 6
10
```

Sample Output 2

Element not found

Code:

```
#include <stdio.h>
int main() {
    int n,a[10],pos,i;
    printf("Enter size of the array: ");
    scanf("%d", &n);
    printf("Enter array elements: ");
    for(i=0; i<n; i++)
        scanf("%d",&a[i]);
    printf("Enter the position at which element is deleted: ");
    scanf("%d", &pos);
    n--;
    for(i=pos; i<=n; i++)
        a[i-1] = a[i];
    printf("After deletion array elements are: ");
    for(i=0; i<n; i++)
        printf("%d\t", a[i]);
    return 0;
}
```

Output:

Output

```
Enter size of the array: 5
Enter array elements: 10 20 30 40 55
Enter the position at which element is deleted: 1
After deletion array elements are: 20 30 40 55
```

D. Seating Arrangement Analysis (2D Arrays)

Objective

Work with multidimensional arrays.

Problem Statement

A classroom has **R rows** and **C columns** of seats.

Each seat contains either:

- 1 → Occupied
- 0 → Empty

Write a program to:

1. Read the seating arrangement.
2. Display the matrix.
3. Count the number of occupied seats.
4. Find which row has the maximum occupied seats.
5. Display the occupancy percentage.

Sample Input

```
3 4
1 1 0 1
0 1 0 0
1 1 1 0
```

Sample Output

Seating Arrangement

1 1 0 1

0 1 0 0

1 1 1 0

Occupied Seats = 7

Row with Maximum Occupancy = 1

Occupancy Percentage = 58.33%

Code:

```
#include <stdio.h>
int main() {
    int R, C;
    printf("Enter number of rows and columns: ");
    if (scanf("%d %d", &R, &C) != 2) return 1;
    int seats[R][C];
    int total_seats = R * C;
    int occupied_seats = 0;
    int max_occupied = 0;
    int max_row_index = 0;
    printf("Enter the seating arrangement (1 for occupied, 0 for empty):\n");
    for (int i = 0; i < R; i++) {
        int current_row_occupied = 0;
        for (int j = 0; j < C; j++) {
            scanf("%d", &seats[i][j]);
            if (seats[i][j] == 1) {
                occupied_seats++;
                current_row_occupied++;
            }
        }
        if (current_row_occupied > max_occupied) {
            max_occupied = current_row_occupied;
            max_row_index = i;
        }
    }
    printf("\nSeating Arrangement\n");
    for (int i = 0; i < R; i++) {
        for (int j = 0; j < C; j++) {
            printf("%d\t", seats[i][j]);
        }
        printf("\n");
    }
    printf("Occupied Seats = %d\n", occupied_seats);
    printf("Row with Maximum Occupancy = %d\n", max_row_index);
    printf("Occupancy Percentage = %.2f%%\n", percentage);
    return 0;
}
```


Output:

Output

```
Enter number of rows and columns: 3 3
Enter the seating arrangement (1 for occupied, 0 for empty):
1 0 0
0 1 0
0 0 1

Seating Arrangement
1 0 0
0 1 0
0 0 1
Occupied Seats = 3
Row with Maximum Occupancy = 0
Occupancy Percentage = 33.33%
```